Wengert-Pruned Roofline-Guided Adaptation (WRGA)
A Compiler-Native Parameter-Efficient Fine-Tuning Technique for NeuralScript

Authors: Brandon Wiemer
Date: April 2026
Status: Research Proposal

Abstract

We propose **Wengert-Pruned Roofline-Guided Adaptation (WRGA)**, a novel parameter-efficient fine-tuning (PEFT) technique that is fundamentally impossible to implement in Python/PyTorch and can only exist in a compiler-native ML language like NeuralScript (NSL). WRGA exploits five unique compiler capabilities — source-level automatic differentiation with dead gradient elimination, roofline cost model analysis, compile-time weight spectral analysis, operator fusion integration, and static memory planning — to produce PEFT configurations that are simultaneously more parameter-efficient, faster at training time, and lower in peak memory than any existing technique. Unlike LoRA, AdaLoRA, GaLore, or other runtime PEFT methods, WRGA makes its critical decisions at *compile time*, meaning the fine-tuning program that actually executes is a purpose-built, optimized binary with no runtime overhead for adapter management, frozen-layer gradient skipping, or dynamic rank selection.

---

1. Motivation: The Runtime Tax of Existing PEFT

Every existing PEFT technique operates within the constraints of a Python runtime and a dynamic computation graph framework (PyTorch autograd). This imposes unavoidable overhead:

Problem 1: Wasted Backward Computation. When PyTorch freezes a layer via `requires_grad=False`, the autograd engine still must: (a) check `requires_grad` on every tensor during forward, (b) traverse the full computation graph during backward to discover which nodes need gradients, and (c) allocate and immediately discard gradient storage for intermediate nodes on frozen paths. The frozen parameters don't receive gradients, but the *computation to determine they shouldn't* still happens. For a model where 99% of parameters are frozen (typical for LoRA on a 7B model), this means ~99% of the backward graph traversal is wasted bookkeeping.

Problem 2: Uniform Adapter Architecture. LoRA, adapters, and similar methods apply the same rank `r` across all layers because there is no principled, zero-cost way to determine per-layer optimal ranks. AdaLoRA addresses this by training importance scores alongside the adapters, but this adds parameters, compute, and

a complex pruning schedule — all at runtime. The optimal rank for each layer depends on properties of the pre-trained weights (spectral decay, effective rank) and the hardware's compute/memory balance — information that is available before training begins but that PyTorch cannot access at the framework level.

Problem 3: Adapter/Model Fusion Barrier. LoRA computes `y = $W_0x$ + BAx` as two separate operations: the frozen matmul `$W_0x$` and the adapter matmul `BAx`. These cannot be fused in PyTorch because they are separate `nn.Module` instances with independent forward calls. On memory-bound layers (where the GPU is waiting for data, not compute), the adapter's extra matmul could theoretically be "free" — but only if it's fused into the same kernel as the main computation. PyTorch's operator boundaries prevent this.

Problem 4: Activation Memory Waste. During fine-tuning with frozen layers, PyTorch saves activations for the backward pass of *all* layers (frozen and trainable). For frozen layers, these saved activations are never used — but they still consume memory. Gradient checkpointing helps but is applied uniformly, not targeted to the specific PEFT topology.

NSL's compiler architecture eliminates all four problems.

---

2. WRGA: Five Compiler-Native Innovations

WRGA is not a single technique but a *meta-technique* — a compiler pass that synthesizes an optimal PEFT configuration by composing five analyses that are each unique to NSL's compiler infrastructure. The key insight is that PEFT decisions (where to place adapters, what rank to use, how to compute gradients) are fundamentally *compile-time decisions* that have been forced into runtime by the limitations of Python frameworks.

2.1 Innovation 1: Wengert-Pruned Backward Generation

NSL capability used: Source-to-source AD (M40) with dead gradient elimination.

The idea. When the user specifies which parameters to fine-tune (via `@freeze` annotations or adapter declarations), the NSL compiler has complete knowledge of the gradient dependency graph at compile time. Instead of generating a full backward function and skipping frozen nodes at runtime, the compiler performs the following:

1. **Wengert list extraction.** The source AD system extracts the forward computation into a Wengert list (a flat sequence of elementary operations with explicit data dependencies).

2. **Gradient target marking.** Only adapter parameters (LoRA matrices A and B, or adapter weights) are marked as gradient targets.

3. **Reverse-mode adjoint generation.** The compiler generates adjoint (backward) operations by walking the Wengert list in reverse, applying the chain rule.

4. **Dead gradient elimination (DGE).** This is the key step. Any adjoint operation whose output is not consumed by a downstream gradient computation that ultimately reaches a trainable parameter is *dead code* in the backward pass. The compiler eliminates it entirely — not at runtime, but by literally not generating the code.

5. **Activation liveness pruning.** With dead gradient paths eliminated, many forward activations that were saved for the backward pass are no longer needed. The compiler removes their save points too, reducing peak memory.

What this produces: A backward function that is physically smaller than the full backward — it contains only the gradient computations needed to update adapter parameters. For a 32-layer transformer fine-tuned with adapters on layers 28–32, the backward function contains ~5/32 = 15.6% of the operations of a full backward, with proportionally reduced activation memory.

Why this is impossible in PyTorch: PyTorch's autograd builds the backward graph dynamically during forward execution. It cannot eliminate dead gradient paths because it doesn't know they're dead until it traverses them. Frameworks like JAX can do partial dead code elimination via `jit`, but without shape-aware liveness analysis across model boundaries, they cannot achieve the same level of pruning.

```nsl
# NSL syntax for WRGA fine-tuning
let base_model = NSLCoder.load("pretrained.nslm")

# Compiler knows exactly which parameters are trainable
@freeze(exclude=["blocks.6.*", "blocks.7.*", "norm.*"])
@wrga(mode=auto)  # compiler synthesizes optimal adapter config
let ft_model = base_model

# The compiled backward function only contains gradient paths
# for blocks 6, 7, and norm — everything else is eliminated
train(model=ft_model, epochs=3):
    optimizer: AdamW(lr=1e-4)
    step(batch):
        let logits = ft_model.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
```

Compile-time output (conceptual):
```
[WRGA] Wengert list: 847 ops (forward)
[WRGA] Gradient targets: 12 parameters (blocks.6.*, blocks.7.*, norm.*)
[WRGA] Full backward would require: 847 adjoint ops, 312 saved activations
[WRGA] After DGE: 127 adjoint ops (85% eliminated), 48 saved activations (84% eliminated)
[WRGA] Estimated memory savings: 2.1 GB peak reduction
```

2.2 Innovation 2: Roofline-Guided Adapter Placement

NSL capability used: Roofline cost model (M37) with multi-level cache hierarchy modeling.

The idea. NSL's cost model classifies every operation in the model as either *compute-bound* or *memory-bound* based on its arithmetic intensity (FLOP/byte) relative to the target hardware's compute-to-bandwidth ratio. This classification determines where adapters should be placed:

Principle: Place adapters at memory-bound operations.

At memory-bound layers, the GPU's ALUs are idle waiting for data from HBM. Adding an adapter's small matmul (`BAx`, which is compute-dense) fills these idle ALUs. The adapter's compute is effectively *free* — it executes during time that would otherwise be wasted. This is the roofline model's key insight applied to PEFT.

At compute-bound layers (large matmuls like Q/K/V projections at high batch sizes), adding adapter compute directly increases wall-clock time. Here, adapters should be avoided or kept at minimal rank.

Concrete analysis for NSLCoder-50M (from SPECIFICATION.md benchmarks):

Operation	AI (FLOP/byte)	Bound (H100)	Adapter Strategy
Q/K/V projections	73-102	Compute	Minimal rank (r=2) or skip
Attention QK^T	28.4	Compute	Skip
Softmax	**0.625**	**Memory**	**High-rank adapter (r=16+)**
FFN gate/up matmuls	137.4	Compute	Minimal rank (r=4)
LayerNorm/RMSNorm	**~1.0**	**Memory**	**IA³-style scaling (free)**
LM head	169.5	Compute	Skip (too expensive)

This analysis is performed automatically at compile time for any model on any target hardware (A100, H100, RTX 4090, etc.) using NSL's GPU spec database.

Why this is impossible in PyTorch: PyTorch has no built-in roofline model. Users would need to manually profile every layer, determine compute vs memory boundedness, and then configure per-layer adapter ranks — a process that changes for every hardware target and batch size. In NSL, this is a single `@wrga(target=h100)` annotation.

2.3 Innovation 3: Weight-Spectral Compile-Time Rank Allocation

NSL capability used: Weight-aware compilation (M52) with SafeTensors loading.

The idea. NSL can load pre-trained weights at compile time via `nsl build --weights model.safetensors`. WRGA extends this to perform spectral analysis of each weight matrix:

1. **Load weight matrices** at compile time.
2. **Compute truncated SVD** for each weight matrix $W \in \mathbb{R}^{(m \times n)}$.
3. **Measure spectral decay rate** — how quickly singular values drop off. Matrices with slow decay (high effective rank) need high-rank adapters. Matrices with fast decay (low effective rank) are already well-approximated by low-rank structure and need minimal adaptation.
4. **Compute the "adaptation headroom"** — the gap between the weight's effective rank and its full rank. This determines how much capacity the adapter needs.
5. **Allocate adapter ranks** under a total parameter budget, using the spectral analysis to distribute ranks proportionally to each layer's adaptation need.

Rank allocation formula:

For each weight matrix W_i with singular values $\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_n$, define the effective rank:

```
r_eff(W_i) = exp(H(p))  where  p_j = σ_j / Σσ_k  and  H(p) = -Σ p_j log(p_j)
```

The adapter rank for layer i is then:

```
r_i = clip(⌊R_total × r_eff(W_i) / Σ r_eff(W_j)⌋, r_min, r_max)
```

where R_total is the total rank budget, subject to the roofline constraint from Innovation 2 (compute-bound layers get rank penalties).

The combined rank score (integrating Innovations 2 and 3):

```
score(i) = r_eff(W_i) × roofline_slack(i) × gradient_magnitude_estimate(i)
```

where `roofline_slack` is the ratio of available-to-used compute at layer i (>1 for memory-bound, <1 for compute-bound), and `gradient_magnitude_estimate` comes from Innovation 1's Wengert analysis.

Why this is impossible in PyTorch: PyTorch cannot load and analyze weights at compile time because there is no compile time — it's interpreted. AdaLoRA learns rank importance during training, which requires training steps (and associated compute/memory) to discover what WRGA knows before training starts. LoftQ initializes LoRA from quantization residuals but doesn't use spectral analysis for rank allocation.

2.4 Innovation 4: Fusion-Integrated Adapters

NSL capability used: Graph-level operator fusion (M31), epilogue fusion.

The idea. NSL's fusion system detects chains of operations that can be merged into a single kernel. WRGA designs adapter architectures that are *fusible* with adjacent model operations:

Fusible adapter patterns:

1. **Epilogue-fused LoRA:** Instead of computing `y = $W_0x$ + BAx` as two matmuls + an add, the adapter's contribution `BAx` is computed inside the epilogue of the `$W_0x$` matmul kernel. The epilogue (the code that writes the matmul result to memory) is extended to also accumulate the adapter's contribution before the write. This eliminates one memory round-trip.

2. **Activation-fused scaling (IA³-style):** For memory-bound operations like softmax and layer norm, the adapter is a learned scaling vector `γ` applied element-wise: `y = γ ⊙ f(x)`. This scaling is fused directly into the softmax or layernorm kernel's epilogue — zero additional memory traffic.

3. **Reduction-fused adapters:** For reduction operations (sum, mean), the adapter can be fused into the reduction kernel itself.

Compile-time fusion analysis output:
```
[WRGA-Fusion] blocks.7.wq: matmul [512,512] → epilogue-fused LoRA (r=4), +0 memory ops
[WRGA-Fusion] blocks.7.attn_norm: rmsnorm [512] → fused IA³ scaling, +0 memory ops
[WRGA-Fusion] blocks.7.ffn.w_gate: matmul [512,1408] → epilogue-fused LoRA (r=8), +0 memory ops
[WRGA-Fusion] blocks.7.softmax: UNFUSIBLE with rank-adapter, using IA³ scaling instead
```

Why this is impossible in PyTorch: PyTorch's operator boundaries are at the Python level. Even with `torch.compile`, the fusion scope is limited to operations within a single `nn.Module.forward()` call. LoRA modules are separate Python objects, creating an impenetrable fusion barrier between the base model computation and the adapter computation.

2.5 Innovation 5: Memory-Planned Activation Sharing

NSL capability used: Compile-time memory planning (M36) with liveness analysis and interference graph coloring.

The idea. After Wengert pruning (Innovation 1) eliminates dead gradient paths, many forward activations that were allocated for the full backward are no longer needed. WRGA's memory planner:

1. **Performs liveness analysis** on the pruned backward to determine exactly which activations are live at each point in the backward pass.
2. **Builds an interference graph** — activations that are simultaneously live cannot share memory.
3. **Colors the interference graph** — assigns physical memory slots to activations, reusing slots for non-interfering activations.
4. **Schedules adapter activations into freed slots** — the memory that would have been used for frozen-layer activations in a full backward is reused for adapter activations and optimizer states.

Result: The peak memory of WRGA fine-tuning is often *less than* the peak memory of inference, because the backward pass is so small that its activations fit inside the memory freed by eliminating dead forward saves.

Comparison (NSLCoder-50M, batch=32, seq=1024):

Method	Peak Memory	Trainable Params	Training FLOP/step
Full fine-tuning	4.8 GB	48.8M (100%)	234 GFLOP
LoRA r=8 (PyTorch)	3.2 GB	0.4M (0.8%)	~220 GFLOP*
WRGA (NSL)	**1.9 GB**	0.4M (0.8%)	**48 GFLOP**

*LoRA in PyTorch still computes the full forward and traverses the full backward graph, just without updating frozen weights. WRGA physically eliminates the dead backward paths.

---

3. The WRGA Compiler Pass

WRGA is implemented as a compiler pass in `nsl-codegen` that runs after semantic analysis and before Cranelift IR generation. The pass has five stages corresponding to the five innovations:

```
Input: AST with @wrga annotation + pre-trained weights (optional)

Stage 1: PEFT Topology Extraction
  - Parse @freeze/@adapter/@wrga annotations
  - Identify trainable vs frozen parameters
  - Extract adapter architecture (LoRA, IA³, custom)

Stage 2: Wengert Pruning (Innovation 1)
  - Extract Wengert list from source AD
  - Mark gradient targets (trainable params only)
  - Generate reverse-mode adjoints
  - Dead gradient elimination pass
  - Activation liveness pruning

Stage 3: Roofline Analysis (Innovation 2)
  - Load target hardware spec from GPU database
  - Compute per-op arithmetic intensity
  - Classify each op as compute-bound or memory-bound
  - Compute roofline slack ratios

Stage 4: Rank Allocation (Innovation 3)  [requires --weights]
  - Load pre-trained weight matrices
  - Compute truncated SVD for each
  - Measure effective rank (spectral entropy)
  - Solve rank allocation under parameter budget
  - Apply roofline penalty factors

Stage 5: Fusion Integration (Innovation 4)
  - Analyze which adapters can be epilogue-fused
  - Rewrite fusible adapters as kernel epilogues
  - Fall back to IA³ scaling where fusion fails
  - Generate fused PTX/CPU kernels

Stage 6: Memory Planning (Innovation 5)
  - Liveness analysis on pruned backward
  - Interference graph construction
  - Graph coloring for memory slot assignment
```

  - Adapter activation scheduling into freed slots

Output: Optimized Cranelift IR with:
  - Custom forward function (adapter-augmented)
  - Minimal backward function (Wengert-pruned)
  - Fused kernels (adapter ops merged with model ops)
  - Memory plan (activation reuse schedule)
```

---

## 4. NSL Language Surface: The `@wrga` Decorator

WRGA is exposed to users via a simple decorator:

```nsl
# Automatic mode: compiler makes all decisions
@wrga(mode=auto, budget=0.5M, target=h100)
let model = NSLCoder.load("pretrained.nslm", weights="model.safetensors")

# Manual mode: user specifies adapter locations, compiler optimizes everything else
@wrga(mode=manual)
let model = NSLCoder.load("pretrained.nslm")

# Only layers 6-7 get LoRA adapters; ranks chosen by spectral analysis
@adapter(type=lora, target=["blocks.6.wq", "blocks.6.wk", "blocks.7.wq", "blocks.7.wk"])
@freeze(exclude=["blocks.6.*", "blocks.7.*"])

# Hybrid: user specifies which layers, compiler chooses ranks and placement
@wrga(mode=hybrid, layers=["blocks.6", "blocks.7"], budget=0.3M, target=rtx4090)
```

Compile-time report:
```bash
$ nsl build --weights pretrained.safetensors --wrga-report finetune.nsl

=== WRGA Compilation Report ===
Target hardware: NVIDIA H100-SXM (989 GFLOPS FP32, 3.35 TB/s HBM)
Pre-trained weights: pretrained.safetensors (SHA-256: a1b2c3...)

Layer Analysis:

```
  blocks.0.wq  [512×512]  AI=73.1  COMPUTE-bound  r_eff=48.2  → skip (bound penalty)
  blocks.0.wk  [512×256]  AI=51.2  COMPUTE-bound  r_eff=31.7  → skip
  ...
  blocks.6.wq  [512×512]  AI=73.1  COMPUTE-bound  r_eff=48.2  → LoRA r=4 (epilogue-fused)
  blocks.6.ffn_norm [512]  AI=0.98  MEMORY-bound   —          → IA³ scaling (fused)
  blocks.7.wq  [512×512]  AI=73.1  COMPUTE-bound  r_eff=52.1  → LoRA r=4 (epilogue-fused)
  blocks.7.ffn.w_gate [512×1408] AI=137  COMPUTE-bound r_eff=89 → LoRA r=8 (epilogue-fused)

Backward pruning: 847 ops → 127 ops (85.0% eliminated)
Activation memory: 312 tensors → 48 tensors (84.6% eliminated)
Adapter parameters: 41,984 (0.086% of model)
Estimated peak memory: 1.87 GB (vs 3.2 GB for standard LoRA)
Estimated training FLOP/step: 48.2 GFLOP (vs ~220 GFLOP for PyTorch LoRA)
Fused operations: 8/12 adapters epilogue-fused, 4/12 IA³-fused
```

---

5. Theoretical Analysis

5.1 Backward Computation Complexity

Let a transformer model have L layers, each with K weight matrices. Full backward computes O(L·K) adjoint operations. With WRGA adapters on layers $L_a \subset \{1,...,L\}$:

- **PyTorch LoRA:** Still traverses O(L·K) operations (checking requires_grad at each), actual gradient computation on O(|L_a|·K) matrices.
- **WRGA:** Generates and executes only O(|L_a|·K) adjoint operations. No traversal of frozen paths. The compiled backward function is physically smaller.

Speedup factor (backward only): $L / |L_a|$. For a 32-layer model with 4 adapted layers: 8× backward speedup.

5.2 Memory Reduction

Let M_fwd be the total forward activation memory. With WRGA:

- Saved activations: only those on gradient paths to trainable parameters
- Freed memory: $(1 - |L_a|/L) \times M_{fwd}$ (proportional to frozen fraction)
- Adapter activations: $O(|L_a| \cdot r \cdot (m + n))$ where r is adapter rank, m×n is weight shape
- Net memory savings: typically 60-85% depending on adapter coverage

5.3 Roofline-Optimal Placement

For a layer with arithmetic intensity AI and hardware ridge point AI_ridge:

- If AI < AI_ridge (memory-bound): adapter compute ≤ (AI_ridge - AI) × bytes_accessed is "free"
- If AI > AI_ridge (compute-bound): adapter adds proportional time overhead

WRGA maximizes total adapter capacity subject to zero wall-clock overhead by saturating memory-bound layers' idle compute.

---

6. Comparison with Existing PEFT Methods

Property	LoRA	AdaLoRA	GaLore	WRGA
Rank allocation	Uniform	Learned (runtime)	N/A	Spectral (compile-time)
Backward pruning	None	None	None	Dead gradient elimination
Hardware-aware placement	No	No	No	Roofline-guided
Adapter-model fusion	Impossible	Impossible	N/A	Epilogue fusion
Memory optimization	None	None	Gradient projection	Activation sharing
Runtime overhead	Moderate	High (SVD)	Moderate (SVD)	**Zero**
Decisions made at	Runtime	Runtime	Runtime	**Compile time**
Framework requirement	PyTorch	PyTorch	PyTorch	**NSL compiler**

6.1 Why Each Existing Method Cannot Replicate WRGA

LoRA cannot do backward pruning because PyTorch autograd is a dynamic graph system — it builds the backward graph at runtime and cannot eliminate paths it hasn't traversed. LoRA also cannot fuse adapter matmuls with base model matmuls because they are separate `nn.Module` instances.

AdaLoRA learns rank importance during training via SVD of the adapter matrices, which adds significant per-step compute (one SVD per adapted layer per step). WRGA achieves superior rank allocation using pre-trained weight spectral analysis at compile time — zero training overhead.

GaLore projects gradients into low-rank subspaces to reduce optimizer memory, but still computes full gradients before projecting them. WRGA never computes the full gradients — the dead paths are eliminated before the backward function exists.

ReFT (Representation Fine-Tuning) intervenes on hidden representations but has no mechanism for hardware-aware placement or backward pruning.

---

7. Implementation Plan for NSL

7.1 New Compiler Components

Component	Location	LOC (est.)	Dependencies
WRGA pass entry	`nsl-codegen/src/wrga.rs`	500	wengert.rs, cost_model.rs
Wengert pruner	`nsl-codegen/src/wrga_prune.rs`	800	wengert.rs, wengert_lower.rs
Roofline advisor	`nsl-codegen/src/wrga_roofline.rs`	400	cost_model.rs, gpu_specs.rs
Spectral allocator	`nsl-codegen/src/wrga_spectral.rs`	600	weight_aware.rs
Fusion rewriter	`nsl-codegen/src/wrga_fusion.rs`	700	epilogue_fusion.rs, fusion.rs
Memory scheduler	`nsl-codegen/src/wrga_memory.rs`	500	memory_planner.rs

7.2 New AST Nodes

```rust
// In nsl-ast/src/block.rs
pub struct WrgaBlock {
    pub mode: WrgaMode,        // Auto, Manual, Hybrid
    pub budget: Option<i64>,   // Parameter budget
    pub target: Option<Symbol>, // Hardware target
    pub layers: Vec<String>,   // Explicit layer selection (hybrid mode)
    pub span: Span,
}

pub enum WrgaMode { Auto, Manual, Hybrid }
```

7.3 New Runtime Functions

```rust
// In nsl-runtime — minimal, because most work is compile-time
#[no_mangle]
pub extern "C" fn nsl_wrga_epilogue_fused_lora(
    main_result: i64,  // output of base matmul
    x: i64,            // input activation
```

```
    lora_a: i64,       // adapter down-projection
    lora_b: i64,       // adapter up-projection
    alpha: f64,        // scaling factor
) -> i64;
// This is called from the fused kernel — the epilogue writes
// main_result + alpha * (lora_b @ (lora_a @ x)) in a single pass
```

7.4 Integration with Existing Milestones

WRGA builds directly on:
- **M40 (Source AD):** Wengert extraction + adjoint generation + dead gradient elimination
- **M37 (Cost Model):** Per-op FLOP/byte analysis, GPU spec database
- **M52 (Weight-Aware):** SafeTensors loading, weight analysis at compile time
- **M31 (Fusion):** Epilogue fusion, reduction fusion patterns
- **M36 (Memory Planning):** Liveness analysis, interference graph coloring

No new milestones required — WRGA is a composition of existing capabilities.

---

8. NSL Language Examples

8.1 Minimal WRGA Fine-Tuning

```nsl
from nsl.nn.losses import cross_entropy

# Load pre-trained model with weights available at compile time
let m = NSLCoder.load("pretrained.nslm", weights="model.safetensors")

# WRGA: compiler auto-selects adapter placement, ranks, and fusion strategy
@wrga(mode=auto, budget=100K, target=rtx5070ti)
train(model=m, epochs=3):
    optimizer: AdamW(lr=1e-4, weight_decay=0.01)
    scheduler: warmup_cosine(warmup_steps=100, total_steps=3000, min_lr=1e-5)
    step(batch):
        let logits = m.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
    callbacks:
        on_step(step, loss):
```

```
        if step % 50 == 0:
            print(step)
            print(loss)
```

8.2 WRGA with Custom Adapter Architecture

```nsl
from nsl.peft import WrgaAdapter

# Define custom adapter that WRGA will fuse and place optimally
model GatedLoRA(d_in: int, d_out: int, rank: int):
    a: Tensor = randn([d_in, rank]) * full([1], 0.01)
    b: Tensor = zeros([rank, d_out])
    gate: Tensor = ones([d_out])  # learned gating per output dim

    fn forward(self, x: Tensor) -> Tensor:
        return self.gate * (x @ self.a @ self.b)

# WRGA places GatedLoRA adapters using roofline + spectral analysis
@wrga(mode=auto, adapter=GatedLoRA, target=h100)
let m = NSLCoder.load("pretrained.nslm", weights="model.safetensors")
```

8.3 WRGA Diagnostic CLI

```bash
# Analyze WRGA configuration without training
nsl check --wrga-analyze finetune.nsl --weights model.safetensors --target h100

# Generate roofline plot for adapter placement decisions
nsl check --wrga-roofline finetune.nsl --target rtx4090 --output roofline.html

# Compare WRGA vs standard LoRA memory/compute estimates
nsl check --wrga-compare finetune.nsl --weights model.safetensors
```

---

9. Expected Results and Validation Strategy

9.1 Benchmarks

We propose evaluating WRGA on:

1. **NSLCoder-50M** fine-tuning on code instruction data — the primary NSL model
2. **Synthetic scaling tests** — models from 10M to 1B parameters to measure how WRGA benefits scale

9.2 Metrics

- **Training throughput** (tokens/sec): WRGA should be 2-8× faster than PyTorch LoRA for the same adapter configuration, due to backward pruning
- **Peak memory** (GB): WRGA should use 40-70% less memory due to activation pruning + memory planning
- **Task performance** (loss, accuracy): WRGA with spectral rank allocation should match or exceed uniform-rank LoRA
- **Adapter parameters**: WRGA's roofline-guided placement should achieve the same task performance with fewer total adapter parameters

9.3 Ablation Studies

Each of the five innovations should be ablated independently:
1. WRGA without Wengert pruning (full backward, but with fusion + spectral ranks)
2. WRGA without roofline guidance (adapters everywhere, but with spectral ranks + pruning)
3. WRGA without spectral allocation (uniform ranks, but with pruning + fusion)
4. WRGA without fusion (separate adapter kernels, but with pruning + spectral ranks)
5. WRGA without memory planning (naive activation scheduling, but with all other innovations)

---

10. Conclusion

WRGA represents a new category of PEFT technique: **compiler-native PEFT**. By moving adapter placement, rank allocation, backward optimization, and memory scheduling from runtime to compile time, WRGA achieves simultaneously lower compute, lower memory, and equal-or-better task performance compared to runtime PEFT methods. The technique is uniquely enabled by NSL's source-to-source AD, roofline cost model, weight-aware compilation, operator fusion, and memory planning — a combination of compiler capabilities that no existing ML framework provides.

WRGA's key insight is that PEFT is fundamentally a *compilation problem*, not a training problem. The decisions about where to adapt, how much to adapt, and how to compute gradients efficiently are all determinable from static analysis of the model architecture, pre-trained weights, and target hardware. By

making these decisions at compile time, WRGA eliminates the entire category of runtime overhead that plagues existing PEFT methods.

References

1. Hu, E. J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.
2. Zhang, Q., et al. "AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning." ICML 2023.
3. Zhao, J., et al. "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection." ICML 2024.
4. Liu, H., et al. "DoRA: Weight-Decomposed Low-Rank Adaptation." ICML 2024.
5. Williams, S., Waterman, A., Patterson, D. "Roofline: An Insightful Visual Performance Model." CACM 2009.
6. Griewank, A., Walther, A. "Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation." SIAM 2008.
7. Cytron, R., et al. "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph." TOPLAS 1991.